

Xiangqi — Relazione di progetto OOP25

Shenghui Hong, Mansour Chabraoui, Haojie Liu, Aicha Nidam

Indice

Capitolo 1 — Analisi.....	2
1.1 Descrizione e requisiti.....	2
1.2 Modello del Dominio.....	3
Capitolo 2 — Design.....	4
2.1 Architettura.....	4
2.2 Design dettagliato.....	6
2.2.1 Shenghui Hong.....	6
2.2.2 Mansour Chabraoui.....	10
2.2.3 Haojie Liu.....	11
2.2.4 Aicha Nidam.....	14
Capitolo 3 — Sviluppo.....	20
3.1 Testing automatizzato.....	20
3.2 Note di sviluppo.....	21
3.2.1 Shenghui Hong.....	21
3.2.2 Mansour Chabraoui.....	22
3.2.3 Haojie Liu.....	22
3.2.4 Aicha Nidam.....	22
Capitolo 4 — Commenti finali.....	23
4.1 Autovalutazione e lavori futuri.....	23
4.1.1 Shenghui Hong.....	23
4.1.2 Mansour Chabraoui.....	23
4.1.3 Haojie Liu.....	23
4.1.4 Aicha Nidam.....	24
Appendice A — Guida utente.....	25
Avvio dell'applicazione.....	25
La scacchiera.....	25
Per effettuare una mossa.....	25
Suggerimenti (Hint).....	26
Timer di gioco.....	27
Situazione di scacco.....	27
Fine della partita.....	27
Salvataggio automatico e ripristino.....	28

Capitolo 1 — Analisi

1.1 Descrizione e requisiti

Il software realizza un'implementazione digitale dello **Xiangqi**, o “scacchi cinesi”, gioco da tavolo strategico a turni per due giocatori, tradizionalmente diffuso in Cina. Ciascun giocatore dispone di sedici pezzi di sette tipologie diverse (Generale, Consigliere, Elefante, Cavallo, Carro, Cannone, Soldato). Ogni tipologia di pezzo è caratterizzata da regole di movimento specifiche, alcune delle quali dipendono anche dalla configurazione corrente della scacchiera. A differenza degli scacchi occidentali, il gioco si svolge su una scacchiera composta da 10 righe e 9 colonne, separata in due metà da un fiume centrale e caratterizzata dalla presenza del Palazzo, un'area composta da 3 righe e 3 colonne.

Descrizione dei pezzi:

- Il Generale rappresenta il pezzo più importante, può muoversi solo verticalmente e orizzontalmente per una cella alla volta;
- I Consiglieri si muovono diagonalmente di una cella;
- Gli Elefanti si muovono diagonalmente di due caselle, e il loro movimento è impedito se la casella intermedia è occupata nella direzione scelta;
- I Carri si muovono orizzontalmente e verticalmente fermandosi prima del primo ostacolo incontrato oppure catturando un eventuale pezzo avversario presente sulla prima casella occupata;
- I Cannoni si muovono come i Carri, ma catturano esclusivamente scavalcando esattamente un altro pezzo;
- I Cavalli si muovono ad L, ma possono essere bloccati da occupazione della casella adiacente sul lato lungo del movimento;
- I Soldati avanzano di una casella senza poter arretrare; possono muoversi anche orizzontalmente dopo aver attraversato il fiume.

Tutti i pezzi possono catturare un pezzo avversario occupandone la casella, purché esso sia raggiungibile secondo le rispettive regole di movimento. Oltre alle regole di movimento dei singoli pezzi, il gioco presenta alcune regole peculiari: il Generale e i Consiglieri sono confinati all'interno del Palazzo, gli Elefanti non possono attraversare il fiume e i due Generali non possono trovarsi sulla stessa colonna senza pezzi intermedi.

Vince la partita chi riesce a mettere sotto scacco matto il Generale avversario; la partita può inoltre concludersi in parità quando nessuno dei due giocatori dispone più di pezzi con capacità offensive.

Il software deve consentire lo svolgimento di una partita completa fra due giocatori, gestendo l'alternanza dei turni, verificando la validità delle mosse, ed il riconoscimento automatico delle condizioni di fine partita. Deve inoltre offrire, come alternativa alla sfida fra due utenti umani, la possibilità di affrontare un avversario artificiale controllato dal software stesso.

Requisiti funzionali

- L'utente deve poter avviare una nuova partita scegliendo fra due modalità: Giocatore contro Giocatore e Giocatore contro Intelligenza Artificiale. In entrambe le modalità il giocatore che utilizza i pezzi rossi muove sempre per primo.
- L'utente deve poter selezionare uno dei propri pezzi ed ottenere in risposta l'insieme di celle verso cui può legalmente muoverlo, tenendo conto delle regole di movimento specifiche del pezzo, degli ostacoli presenti sulla scacchiera e delle regole generali del gioco (ad esempio, un giocatore non può effettuare una mossa che lasci il proprio Generale sotto scacco, né una mossa che porti i due Generali a fronteggiarsi direttamente sulla stessa colonna senza pezzi nel frammazzo).
- Il software deve rilevare automaticamente le condizioni di scacco, di scacco matto e di parità, notificandole all'utente e, nei primi due casi, terminando opportunamente la partita in caso di scacco matto.
- Nella modalità contro l'Intelligenza Artificiale, il software deve calcolare autonomamente la mossa dell'avversario artificiale ad ogni suo turno.
- L'utente deve poter richiedere un numero limitato di suggerimenti (hint) per partita, ricevendo indicazione di una mossa ritenuta vantaggiosa.
- L'utente deve poter salvare lo stato di una partita in corso ed, in un momento successivo, ripristinarla esattamente da dove era stata interrotta.
- Ciascun giocatore deve rispettare un tempo massimo per la singola mossa ed un tempo massimo complessivo per l'intera partita; l'esaurimento di uno dei due tempi comporta la sconfitta immediata del giocatore.

Requisiti non funzionali

- L'interfaccia utente deve essere interattiva e fornire un riscontro tempestivo alle azioni dell'utente (selezione dei pezzi, esecuzione delle mosse, aggiornamento dei tempi residui).
- Lo stato di una partita salvata deve poter essere conservato anche dopo la chiusura dell'applicazione, per essere ripristinato in un'esecuzione successiva.
- Il software deve essere portabile su qualunque piattaforma dotata di un ambiente di esecuzione compatibile.

1.2 Modello del Dominio

Il dominio dello Xiangqi ruota attorno al concetto della **scacchiera** (*Board*), una griglia di posizioni sulla quale vengono disposti i **pezzi** (*Piece*). Ogni pezzo appartiene a uno dei due **giocatori** (*Player*), contraddistinti da un **colore** (*Color*), e ciascuna occupa una **posizione** (*Position*) sulla scacchiera, definita da una coppia di coordinate riga e colonna.

Inoltre, ogni pezzo è caratterizzato da un **tipo** (*PieceType*), che ne determina le regole di movimento. In base al proprio tipo e alla situazione della scacchiera, ogni pezzo riesce a ricavarne l'insieme delle **mosse** (*Move*) che potrebbe compiere: una mossa è semplicemente la coppia costituita da una posizione di partenza ed una di arrivo.

Tutte queste informazioni vengono salvate all'interno del **modello di gioco** (*GameModel*), in cui viene specificato anche la **modalità di gioco** (*GameModeType*). Un'alternativa per conoscere la modalità di gioco si può ricorrere alla proprietà *isHuman()* dei giocatori. Infatti la modalità PVE

è caratterizzata da un giocatore umano e un giocatore bot.

I controlli di scacco, scacco matto e patta vengono affidati ad una classe apposta che assume il ruolo di un **sistema di controllo delle regole** del gioco (*RuleEngine*). Inoltre, conoscendo le informazioni della scacchiera e avendo tutti i controlli necessari, fornisce la lista delle mosse legali per un determinato pezzo.

Gli elementi costitutivi del problema sono sintetizzati in Figura 1.1.

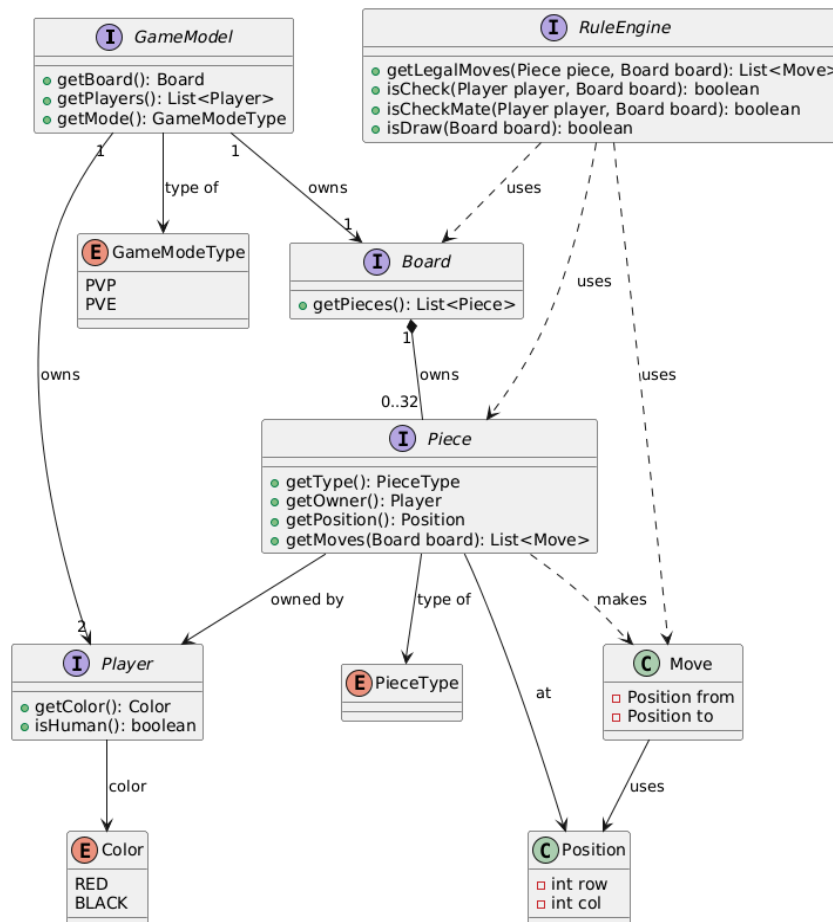


Figura 1.1: Modello del Dominio.

Capitolo 2 — Design

2.1 Architettura

L'architettura del sistema Xiangqi segue il pattern architetturale **MVC (Model-View-Controller)**, adottando una separazione tra la gestione dello stato del gioco (**Model**), la gestione degli eventi e del flusso applicativo (**Controller**) e la rappresentazione grafica dell'interfaccia utente (**View**).

Il **Model** è il componente responsabile della gestione della logica e dello stato interno della partita. L'interfaccia *GameModel* fornisce al resto del sistema un punto di accesso alle informazioni e alle operazioni che riguardano lo stato attuale della partita. La logica relativa ai

singoli elementi del gioco è inoltre distribuita tra altre componenti, come l'interfaccia *Piece*, che definisce il comportamento dei diversi tipi di pedine e permette di calcolare le mosse valide in base alla *Board* corrente.

Il **Controller** rappresenta il punto di coordinamento tra modello e vista. L'interfaccia *GameController* gestisce il flusso generale della partita, occupandosi di operazioni come l'avvio del gioco, la selezione delle pedine o l'applicazione delle mosse. Il controller riceve gli eventi generati dalla vista tramite l'interfaccia *InputHandler*, che funge da elemento di collegamento tra *GameView* e *GameController*. Il controller delega quindi al Model tutte le operazioni che riguardano la logica di gioco e, una volta ottenuto il nuovo stato della partita, richiede alla View l'aggiornamento della rappresentazione grafica.

La **View** è rappresentata dall'interfaccia *GameView*, che definisce tutte le operazioni necessarie per aggiornare la rappresentazione grafica del gioco. La vista riceve informazioni dal controller e si occupa esclusivamente della loro rappresentazione.

All'interno del sistema è presente inoltre il componente *MoveCalculator*, che rappresenta un servizio separato dedicato al calcolo delle mosse suggerite dall'intelligenza artificiale. Questo componente utilizza le informazioni fornite dal *GameModel* per simulare possibili situazioni di gioco e determinare la mossa migliore per il giocatore corrente, senza modificare direttamente lo stato della partita.

Grazie alla separazione della responsabilità indotta dall'architettura MVC, è possibile sostituire completamente l'interfaccia grafica senza modificare il modello o il controller, oppure introdurre nuovi algoritmi di calcolo delle mosse modificando solamente il componente *MoveCalculator*.

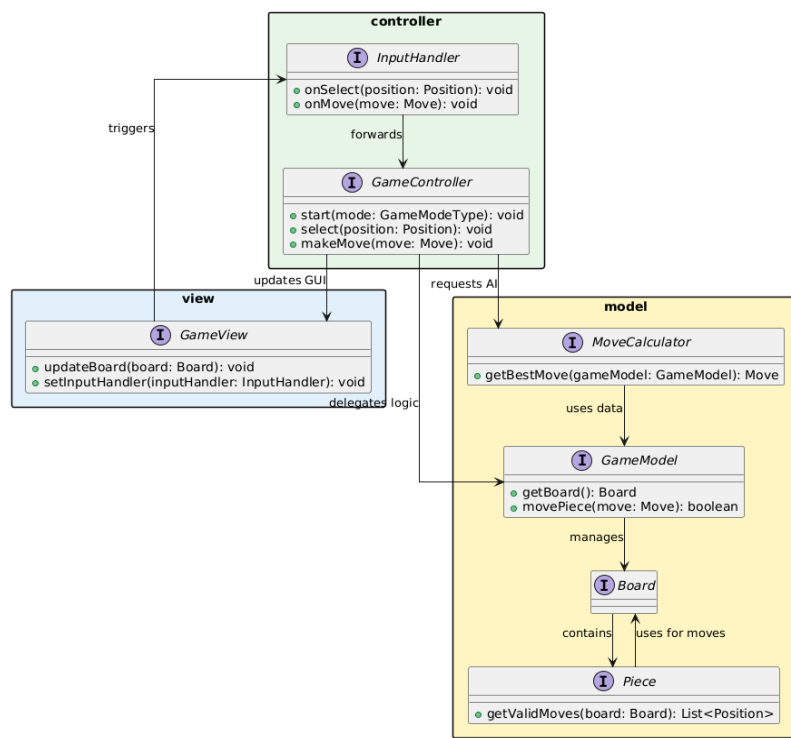


Figura 2.1: Rappresentazione UML per l'architettura MVC del sistema.

2.2 Design dettagliato

2.2.1 Shenghui Hong

Gerarchia dei pezzi di gioco

Problema: Lo Xiangqi prevede sette tipologie di pezzi (Generale, Consigliere, Elefante, Cavallo, Carro, Cannone e Soldato). Tutti condividono lo stesso stato, costituito dal proprietario, dalla posizione corrente e dal valore associato al pezzo, nonché la necessità di verificare, per ogni possibile destinazione, se questa ricada all'interno dei limiti della scacchiera e se la casella sia libera, occupata da un pezzo avversario oppure da un pezzo alleato. Ciò che distingue le diverse tipologie è esclusivamente l'algoritmo utilizzato per generare le possibili destinazioni (ad esempio il passo singolo del Soldato, il movimento a "L" del Cavallo o lo scorrimento lungo una direzione del Carro e del Cannone). Era pertanto necessario evitare la duplicazione della logica comune mantenendo indipendente l'implementazione delle regole specifiche di ciascun pezzo.

Soluzione: La classe astratta *AbstractPiece* implementa l'interfaccia *Piece* e raccoglie tutto lo stato condiviso dai pezzi, oltre alla logica comune necessaria per verificare la validità di una destinazione. In particolare, il metodo *tryAddMove()* centralizza i controlli relativi ai limiti della scacchiera, alla presenza di pezzi alleati o avversari e alla gestione delle catture, comunicando alle sottoclassi se la generazione delle mosse possa proseguire nella stessa direzione. Ogni sottoclasse concreta (*Soldier*, *Horse*, *Elephant*, *Advisor*, *General*, *Chariot* e *Cannon*) implementa il metodo *getMoves()*, occupandosi unicamente della generazione delle destinazioni secondo le regole di movimento proprie del relativo pezzo e delegando alla superclasse tutti i controlli comuni. Inoltre, i costruttori delle sottoclassi sono dichiarati *protected*, demandando la creazione delle istanze alla *PieceFactory*, responsabile dell'inizializzazione corretta dei diversi tipi di pezzi. Questa soluzione riduce la duplicazione del codice, separa chiaramente la logica comune da quella specifica dei singoli pezzi e rende il sistema più semplice da mantenere ed estendere, poiché eventuali modifiche alle regole condivise sono concentrate in un unico punto, mentre l'aggiunta di una nuova tipologia di pezzo richiederebbe esclusivamente l'implementazione del relativo algoritmo di generazione delle mosse.

Alternativa possibile: Un'alternativa avrebbe previsto l'utilizzo del pattern **Strategy**, mantenendo un'unica implementazione di *Piece* e delegando la logica di movimento ad una *MovementStrategy*. Tale soluzione avrebbe favorito la composizione rispetto all'ereditarietà, ma non avrebbe apportato benefici concreti nel contesto del progetto: il comportamento di movimento di un pezzo rimane invariato per tutta la durata della partita e non necessita di essere sostituito dinamicamente. Si è quindi preferita la soluzione basata su *AbstractPiece*, più semplice e adeguata ai requisiti.

Limite noto della soluzione: L'interfaccia *Piece* espone anche i metodi *getCurrentValue()* e *setValue(int)*, utilizzati esclusivamente dalla classe dell'intelligenza artificiale per la valutazione delle mosse. Questa scelta introduce una contaminazione tra il modello di dominio e la logica di valutazione, poiché il punteggio non rappresenta una proprietà intrinseca del pezzo. Sono state considerate due possibili evoluzioni: mantenere tali valori in una struttura dati locale al *MoveCalculatorImpl* (ad esempio una *Map<Piece, Integer>*), oppure introdurre un *ScoredPiece* basato sul pattern **Decorator**, che aggiunga il punteggio senza modificare il modello di dominio.

L'uso della mappa o del pattern Decorator avrebbe garantito una migliore separazione tra modello di dominio e logica di valutazione, ma avrebbero richiesto una rifattorizzazione dell'attuale implementazione dell'intelligenza artificiale. Considerata la semplicità dell'algoritmo di valutazione e il limitato beneficio pratico rispetto ai requisiti del progetto, si è scelto di mantenere l'implementazione esistente e di lasciare questa modifica come possibile evoluzione futura.

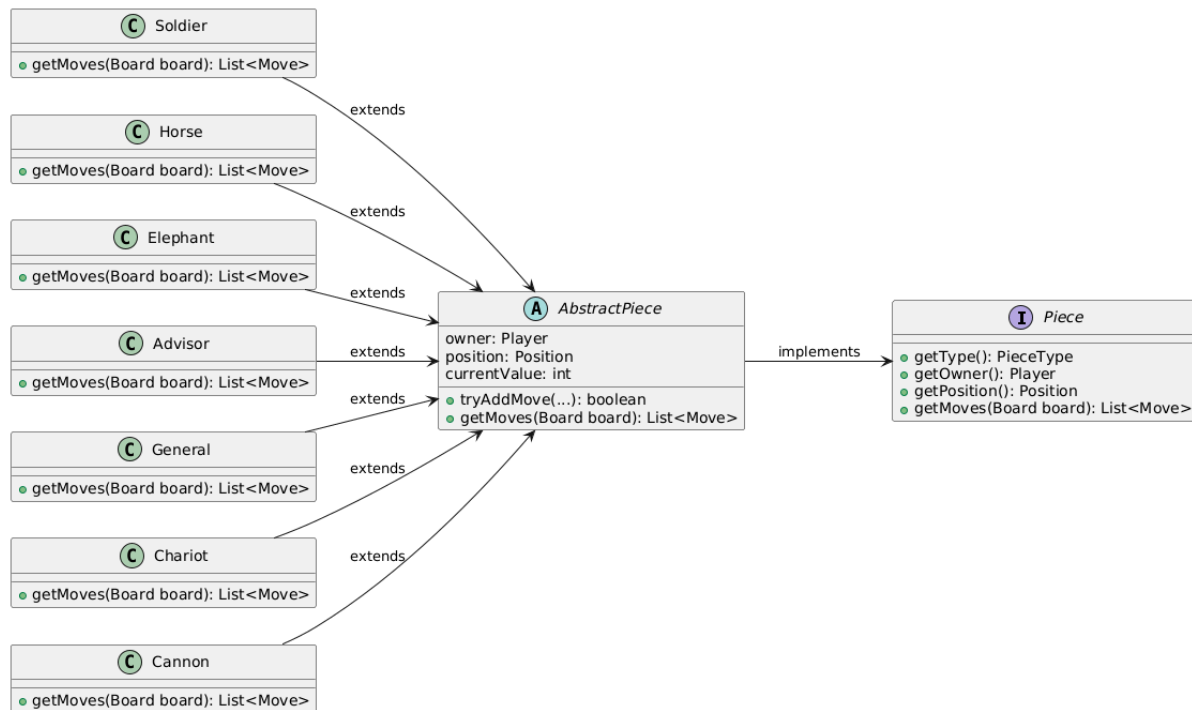


Figura 2.2: Rappresentazione UML del pattern Template per la gerarchia dei pezzi.

Pattern utilizzato: Template Method. Il metodo *tryAddMove()* è il passo invariante dell'algoritmo (definito in *AbstractPiece* e riutilizzato da quasi tutte le sottoclassi), mentre *getMoves()* è il metodo variante che ciascuna sottoclasse concreta ridefinisce per esprimere la propria geometria di movimento.

Creazione centralizzata dei pezzi

Problema: Le istanze dei pezzi devono essere create in diversi contesti dell'applicazione: durante l'inizializzazione di una nuova partita, nella copia dello stato di gioco utilizzata dall'intelligenza artificiale per simulare le mosse e nella ricostruzione di una partita salvata. In tutti questi casi i componenti che richiedono la creazione di un pezzo non dovrebbero dipendere dalla conoscenza delle classi concrete corrispondenti ai diversi tipi di pezzo, così da mantenere basso l'accoppiamento tra i componenti del sistema.

Soluzione: La classe *PieceFactory* centralizza l'intera responsabilità della creazione dei pezzi, incapsulando la corrispondenza tra ciascun *PieceType* e la relativa implementazione concreta. A tale scopo espone tre metodi pubblici dedicati ai principali scenari d'uso: l'inizializzazione della

configurazione iniziale della partita (*initializePieces()*), la copia profonda di un pezzo (*copyPiece()*) e la ricostruzione di un pezzo a partire da una rappresentazione persistente (*fromStoredPiece()*). Le classi concrete dei pezzi non sono quindi istanziabili direttamente dall'esterno, demandando alla factory il controllo dell'intero processo di creazione.

Questa soluzione concentra in un unico punto tutta la logica di istanziazione, evitando che altri componenti del sistema dipendano dalle implementazioni concrete dei pezzi. In questo modo il codice risulta più modulare e manutenibile, poiché eventuali modifiche alle modalità di creazione o l'introduzione di nuove tipologie di pezzi richiedono interventi esclusivamente all'interno della factory.

Alternativa possibile: Per la copia dei pezzi si sarebbe potuto adottare il pattern **Prototype**, introducendo un metodo *copy()* nell'interfaccia *Piece*, implementato polimorficamente da ciascuna sottoclasse. Si è preferito riutilizzare *PieceFactory*, mantenendo un unico punto di conoscenza della corrispondenza tra *PieceType* e relativa implementazione concreta, già utilizzato anche per l'inizializzazione della partita e per il ripristino da salvataggio. In questo modo si evita l'introduzione di un secondo meccanismo di creazione e si mantiene l'interfaccia *Piece* focalizzata esclusivamente sulle responsabilità del dominio, senza aggiungervi funzionalità di natura tecnica. La soluzione adottata garantisce inoltre una maggiore coerenza, poiché tutte le modalità di creazione dei pezzi sono centralizzate all'interno della factory.

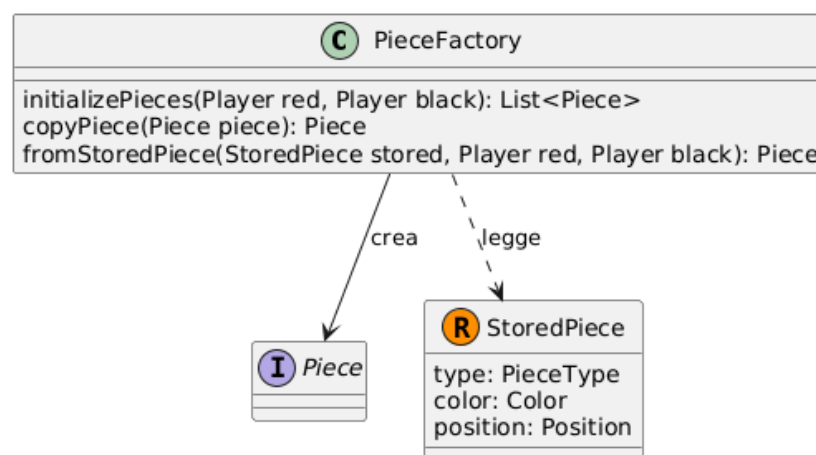


Figura 2.3: Rappresentazione UML del pattern Factory per la creazione dei pezzi.

Pattern utilizzato: Factory (nella variante *static factory*): *PieceFactory* incapsula la logica necessaria per determinare quale classe istanziare in base al tipo richiesto, disaccoppiando i codici clienti dalle classi concrete della gerarchia dei pezzi.

GameModel come radice del modello e le sue istantanee per la simulazione

Problema: Il sistema necessita di un unico punto di accesso che rappresenti lo stato complessivo della partita comprendendo la modalità di gioco, i giocatori, il turno corrente, il numero di suggerimenti disponibili e lo stato della partita). Allo stesso tempo, alcune componenti della logica di gioco, come il *RuleEngine* e il *MoveCalculator*, devono poter simulare l'applicazione di un mossa per valutarne gli effetti senza modificare lo stato reale della partita.

Soluzione: *GameModel* rappresenta il punto di ingresso del model nell'architettura MVC e svolge il ruolo di coordinatore dello stato della partita. Esso aggrega la *Board*, i giocatori e tutte le informazioni necessarie allo svolgimento della partita, occupandosi delle operazioni di alto livello, quali l'inizializzazione del gioco, la gestione dei turni, il conteggio degli hint e la gestione dello stato. Per consentire la simulazione di mosse ipotetiche, *GameModel* mette a disposizione il metodo *copyState()*, che produce un'istanza di *GameState* completamente indipendente dal modello reale. La nuova scacchiera viene costruita creando nuove istanze di tutti i pezzi ottenuti tramite il metodo *copyPiece()* della classe *PieceFactory*, evitando qualsiasi condivisione di oggetti con l'originale. Il modello reale resta l'unica fonte di verità della partita, mentre l'istantanea si comporta come un ambiente isolato usa e getta su cui è sicuro applicare mosse ipotetiche senza produrre effetti collaterali sullo stato della partita.

Nota del Design: Deliberatamente, il metodo *movePiece()* di *GameModel* non effettua alcuna verifica di legalità della mossa, ma assume che questa sia già stata validata a monte tramite *RuleEngine*. Questa scelta separa chiaramente le responsabilità dei due componenti: *RuleEngine* si occupa della validazione delle regole e della determinazione delle mosse consentite, mentre *GameModel* si occupa esclusivamente dell'esecuzione della mossa e dell'aggiornamento dello stato della partita. In questo modo si evita di accoppiare il modello alla logica di validazione e, di conseguenza, si evita anche una dipendenza circolare tra *GameModel* e *RuleEngine*, dato che quest'ultimo dipende già da *Board* e *Piece* rispettando il principio di **Single Responsibility**.

Il rovescio della medaglia è che *movePiece()* si basa su un contratto espresso solo tramite documentazione e convenzione d'uso, non garantito dal sistema dei tipi di Java: nulla impedisce, a livello sintattico, che un chiamante invochi direttamente *movePiece()* passando una mossa illegale, aggirando il flusso previsto dall'architettura MVC. Il corretto funzionamento del sistema dipende quindi dal rispetto di un contratto puramente architetturale: ogni mossa deve essere validata dal *RuleEngine* prima di essere applicata dal *GameModel*, e questa responsabilità ricade interamente su *GameController*, l'unico componente che orchestra entrambe le chiamate in sequenza.

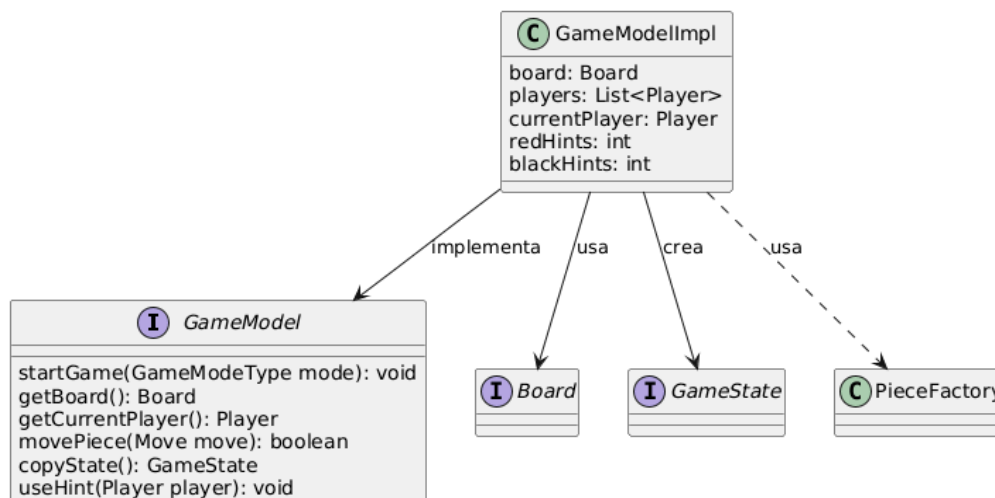


Figura 2.4: Rappresentazione UML del *GameModel* come radice del modello.

Pattern/principio utilizzato: *GameModel* è l'unico punto di accesso allo stato della partita per i livelli superiori dell'applicazione. La produzione di copie indipendenti tramite *copyState()* è riconducibile all'idea di *defensive copying*, consentendo di esplorare stati ipotetici senza modificare il modello originale.

2.2.2 Mansour Chabraoui

Eccessiva responsabilità della classe View principale

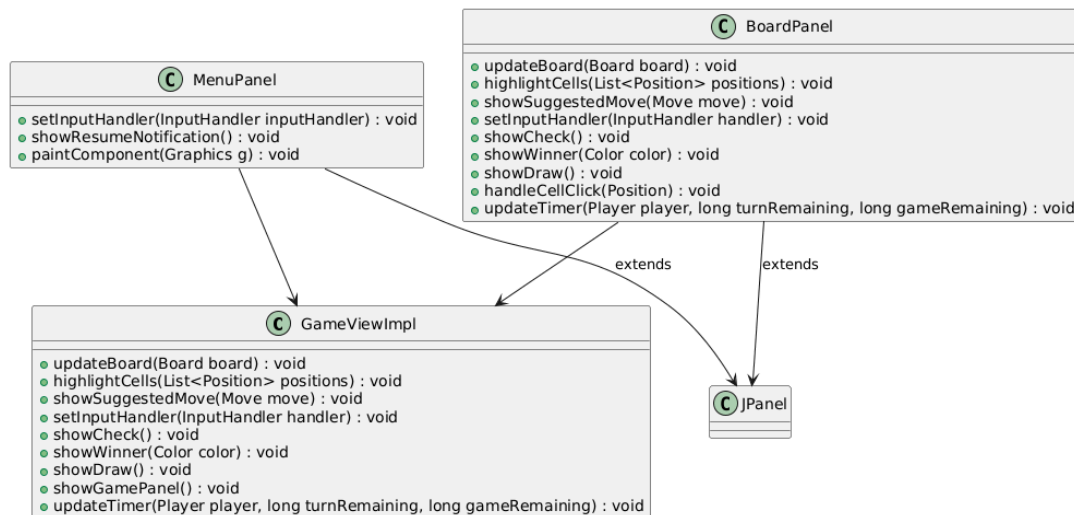


Figura 2.5:

Rappresentazione UML della decomposizione di *GameViewImpl* tramite componenti Swing.

Problema: durante lo sviluppo dell'interfaccia grafica, la classe *GameViewImpl* aveva il compito di gestire l'intera GUI dell'applicazione, includendo sia la schermata principale del menu sia la schermata della partita. Questa responsabilità risultava eccessiva, portando ad una classe difficilmente manutenibile e poco coerente con il principio di singola responsabilità.

Soluzione: Per ridurre la complessità della classe *GameViewImpl*, la gestione dell'interfaccia grafica è stata suddivisa in componenti specializzati. In particolare, sono state introdotte le classi *MenuPanel* e *BoardPanel*, entrambe estensioni di *JPanel*, incaricate rispettivamente della gestione della schermata di menu e della visualizzazione e aggiornamento della board di gioco. La classe *GameViewImpl* mantiene il ruolo di coordinatore della vista, delegando alle componenti interne la gestione degli elementi grafici specifici. La suddivisione ulteriore di *BoardPanel* è stata scartata, mantenendo all'interno della stessa classe le funzionalità strettamente correlate alla visualizzazione e all'interazione con la board. Questa scelta permette di preservare una maggiore coesione interna, riducendo il numero di dipendenze tra componenti grafiche.

2.2.3 Haojie Liu

Simulazione dell'intelligenza artificiale

Problema: Per poter implementare la modalità PVE e il sistema di suggerimento delle mosse, sorge la necessità di implementare un sistema che calcola la mossa migliore in un istante di tempo preciso. La scelta è inoltre limitata dallo stato della scacchiera e dei pezzi su di essa.

Soluzione: Per risolvere il problema, viene utilizzato un sistema a punteggio, ovvero vengono assegnati dei valori ad ogni pezzo in base:

- al suo tipo: il generale ha il punteggio massimo;
- alla sua posizione sulla scacchiera: i valori dei pezzi vengono aumentati o diminuiti al verificarsi di determinate condizioni;
- al cambiare del numero di pezzi presenti sulla scacchiera: il valore di alcuni pezzi dipendono dal numero di pezzi rimanenti sulla scacchiera;
- all'insieme dei pezzi che sta minacciando o proteggendo.

Il calcolo del punteggio viene suddiviso in più fasi, ciascuna è implementata da una funzione dedicata.

1. Nella prima fase, prima di effettuare la mossa, viene verificato se si tratta di una mossa di cattura; in caso viene assegnato un bonus per attacco proporzionale al valore del pezzo catturato.
2. Nella seconda fase, viene generata una copia dello stato di gioco *GameState* sulla quale la mossa viene applicata. Sulla scacchiera simulata viene assegnato un punteggio della posizione ad ogni pezzo, in funzione della propria posizione e di eventuali condizioni specifiche, valutate in modo indipendente dagli altri pezzi.
3. Nella fase successiva, vengono calcolate le interazioni con gli altri pezzi. Per ciascun pezzo viene verificato se sta minacciando un pezzo avversario, se il bersaglio è protetto da altri pezzi, e se il pezzo stesso sta proteggendo un pezzo alleato e se l'azione offensiva lo espone a un rischio.
Ogni pezzo ottiene un valore per ogni situazione verificata, aggiornando il suo valore corrente.
4. Infine, viene calcolato il punteggio complessivo della configurazione simulata sommando i valori dei pezzi appartenenti al giocatore che sta effettuando la valutazione. Successivamente, il risultato viene confrontato con il punteggio migliore ottenuto fino a quel momento e, se superiore, vengono aggiornati sia il punteggio massimo sia la mossa corrispondente. Dopo aver valutato e simulato tutte le mosse candidate, l'algoritmo restituisce la mossa associata al punteggio più elevato.

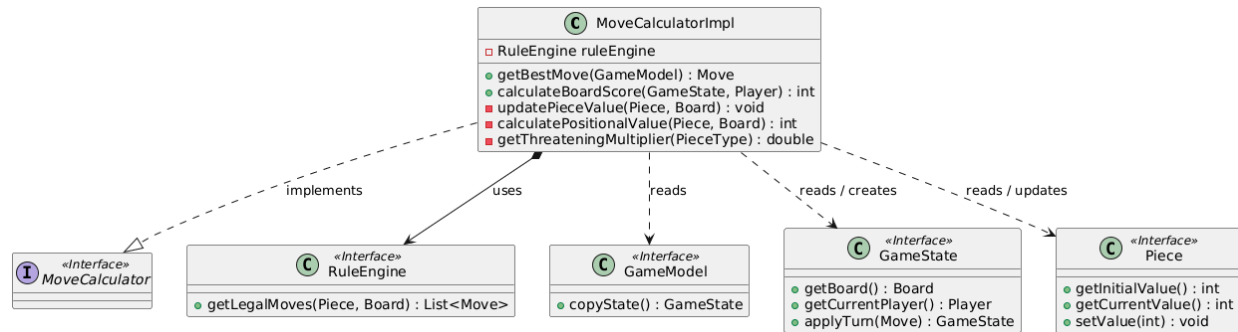


Figura 2.6: Rappresentazione UML per la simulazione dell'intelligenza artificiale.

Tutti i valori in dettaglio sono definite in [MoveCalculatorImpl.java:19](#).

Concetto di valore posizionale

Problema: Quando un pezzo A sta proteggendo un pezzo B, il pezzo A ottiene il valore del pezzo B, come valore aggiuntivo al valore proprio di A. Nel caso in cui un pezzo C sta minacciando il pezzo A, allora il suo valore finale dipende dal valore di A, che è ulteriormente dipendente dal valore del pezzo B. Questo genera una dipendenza circolare tra i punteggi dei pezzi, rendendo difficile ottenere una valutazione coerente e deterministica.

Soluzione: Per eliminare questa dipendenza viene introdotto un valore posizionale, determinato esclusivamente dalla sua posizione sulla scacchiera e da condizioni locali indipendenti dagli altri pezzi. In questo modo ogni contributo è basato su valori stabili e indipendenti, evitando dipendenze circolari durante il calcolo del punteggio complessivo.

Bilanciamento della funzione di valutazione

Problema: Una funzione di valutazione che assegni lo stesso peso alle azioni offensive e difensive tende spesso a privilegiare mosse eccessivamente conservative, penalizzando comportamenti aggressivi che risultano spesso vantaggiosi nel gioco.

Soluzione: Per orientare l'intelligenza artificiale verso strategie più efficaci, le azioni offensive vengono premiate mediante coefficienti moltiplicativi superiori a quelli assegnati alle azioni difensive.

In particolare, le azioni di catture e minacce ricevono un fattore moltiplicativo che viene moltiplicato al valore posizionale del pezzo catturato/minacciato per la valutazione dei punteggi. Inoltre, viene verificato che lo scambio sia conveniente confrontando il valore posizionale del pezzo attaccante con quello del pezzo coinvolto, evitando di incentivare catture sfavorevoli.

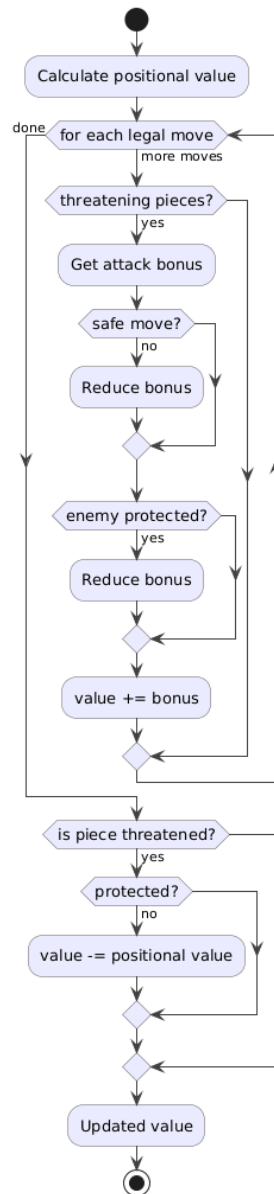


Figura 2.7: Diagramma di flusso sintetico dell'algoritmo di valutazione utilizzato.

Implementazione del timer di gioco usando i thread

Problema: Per essere conformi al gioco reale, sorge la necessità di aggiungere un timer per ciascun giocatore, composto da due valori, uno per il turno corrente, e l'altro per l'intera partita. Tale timer, deve aggiornarsi in tempo reale ed inoltre non deve bloccare il flusso del resto del gioco.

Soluzione: Essendo il timer un componente che non dovrebbe bloccare le azioni sugli altri componenti, ma un processo eseguito in modo sincrono, si fa uso ai thread, ovvero rendere il

processo del timer eseguibile in contemporanea agli altri processi. Così facendo, il timer si aggiorna senza bloccare il resto del gioco.

2.2.4 Aicha Nidam

Confine tra interfaccia utente e logica di gioco

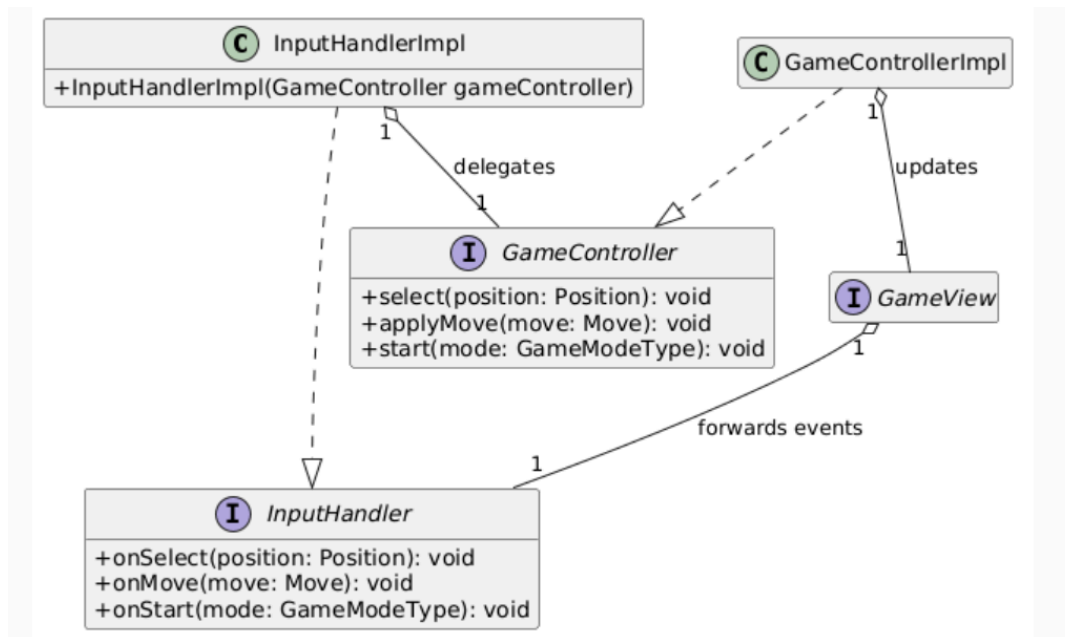


Figura 2.8:

Diagramma UML che mostra l'interazione tra *GameView*, *InputHandler* e *GameController*.

Problema: La GUI genera eventi grezzi ed eterogenei (un click su una cella, la pressione del bottone di suggerimento, la richiesta di avviare o riprendere una partita...). Se questi eventi venissero gestiti all'interno della View, la logica del gioco finirebbe accoppiata ai dettagli di implementazione grafica. Allo stesso tempo lasciare che la GUI comunicasse direttamente con il *GameController* significherebbe mescolare due vocabolari diversi in un solo componente: quello degli eventi utente e quello delle azioni di dominio.

Soluzione: Sono introdotti due componenti distinti che separano le responsabilità tra View e la logica applicativa:

- *InputHandler*: rappresenta il punto di ingresso di tutti gli eventi provenienti dalla View. Esso traduce gli eventi grezzi generati dall'interfaccia utente nelle operazioni di dominio corrispondenti, delegando l'esecuzione al *GameController*.
- *GameController*: coordina *GameModel*, *RuleEngine*, *MoveCalculator* e *GameLoader* per decidere cosa succede realmente dopo ogni azione, ed è l'unico punto dal quale si aggiorna la View, per garantire una netta separazione tra logica applicativa e interfaccia grafica.

Questa struttura segue il pattern MVC nella variante in cui la view non osserva direttamente il model tramite **Observer**, ma riceve aggiornamenti espliciti dal controller. Il *GameModel* non conosce l'esistenza di *GameView*, e potrebbe così essere pilotato da qualsiasi altra interfaccia. *InputHandler* non introduce un nuovo ruolo, ma svolge quello di **Adapter** all'interno del boundary utente: il target del pattern è *GameController*, l'adapter è *GameView*, e la conversione avviene metodo per metodo.

Coordinamento centralizzato del flusso di gioco

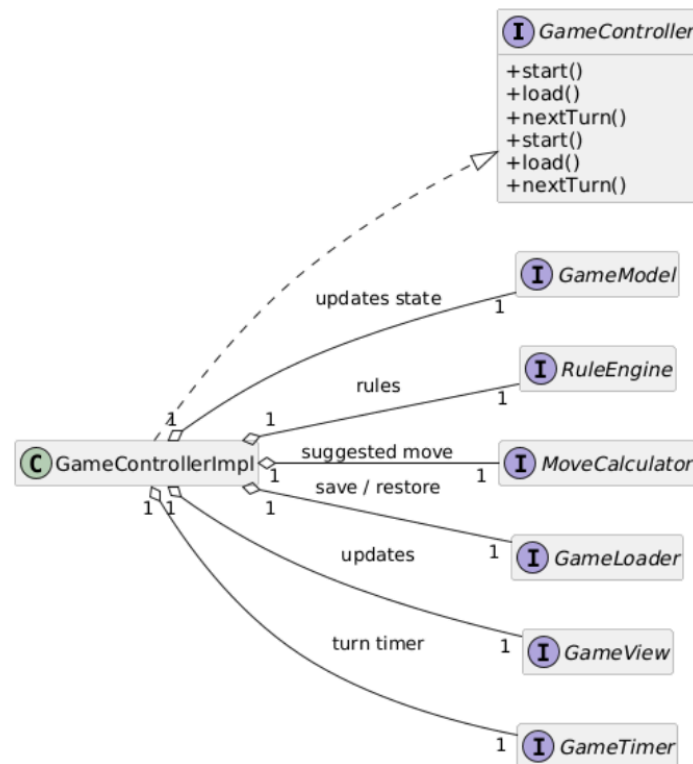


Figura 2.9: Diagramma UML del *GameController* come coordinatore centralizzato.

Problema: Il flusso di una partita comprende diversi momenti di transizione: avvio di una nuova partita, ripristino di un salvataggio, conclusione del turno corrente e passaggio al giocatore successivo. Sebbene questi eventi abbiano origini differenti, richiedono sempre la stessa sequenza di operazioni: aggiornare la vista, verificare le condizioni di terminazione della partita (patta o scacco matto), notificare eventuali stati particolari (ad esempio lo scacco), e infine decidere se il turno successivo appartenga a un giocatore umano o al bot. Una possibile soluzione consisteva nel separare tali responsabilità introducendo componenti dedicati, ad esempio un *TurnManager* o gestori distinti per il turno umano e quello automatico. Questa soluzione avrebbe però distribuito la conoscenza del ciclo di vita della partita tra più classi, aumentando l'accoppiamento, e il rischio di duplicare la logica di coordinamento, con conseguente maggiore difficoltà nel mantenere il flusso di gioco coerente durante l'evoluzione del software.

Soluzione: Si è scelto di concentrare il coordinamento del ciclo di vita della partita nel *GameController*, delegando invece tutte le responsabilità specialistiche ai componenti dedicati. Il metodo privato *dispatchTurn()* rappresenta il punto di convergenza per le transizioni della partita: viene invocato sia all'avvio (*start()*), sia dopo il ripristino di un salvataggio (*load()*), sia al termine di ogni turno (*nextTurn()*). Al suo interno vengono eseguite tutte le operazioni comuni, e soltanto nella fase finale il flusso viene differenziato tra *playerTurn()* e *botTurn()* in base al giocatore corrente e la modalità di gioco scelta. Rispettando il principio *Don't Repeat Yourself (DRY)*, l'aggiunta di una nuova regola di fine partita, di una nuova notifica alla vista o di un'ulteriore operazione da eseguire ad ogni cambio turno richiede la modifica di un solo metodo, riducendo il rischio di inconsistenze durante l'evoluzione del software. La stessa struttura ha consentito di integrare naturalmente funzionalità come il sistema di *hint()*. La richiesta di un suggerimento costituisce infatti un evento del flusso di gioco e viene gestita dal controller, che verifica la disponibilità degli hint, coordina l'aggiornamento del modello e della vista e delega il calcolo della mossa suggerita al *MoveCalculator*. Il *GameController* stabilisce quando ciascun componente debba intervenire, senza mai occuparsi di come eseguire l'operazione. Le regole sono delegate al *RuleEngine*, il calcolo delle suggerimento al *MoveCalculator*, la persistenza al *GameLoader* e l'evoluzione dello stato al *GameModel*. In questo modo il controller mantiene un'unica responsabilità di alto livello: coordinare le transizioni della partita, configurandosi come punto centrale del ciclo di vita della partita.

Separazione tra geometria del movimento e regole del gioco

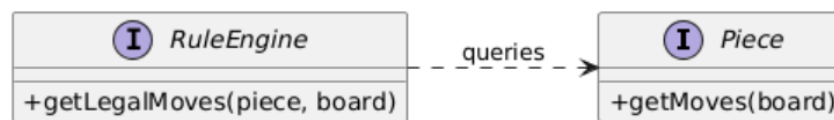


Figura 2.10:

Diagramma UML che mostra la separazione delle responsabilità tra *Piece* e *RuleEngine*.

Problema: Nello Xiangqi ogni pezzo possiede regole di movimento proprie, mentre la legalità di una mossa dipende anche dallo stato complessivo della partita. Ad esempio, un Carro può muoversi geometricamente lungo una colonna libera, ma tale movimento non è consentito se lascia il proprio Generale sotto scacco o viola la regola del Flying General. Attribuire entrambe le responsabilità alle classi *Piece* avrebbe introdotto un forte accoppiamento tra i singoli pezzi e le regole globali del gioco, obbligando ogni implementazione a conoscere aspetti che non appartengono al proprio dominio.

Soluzione: Sono stati applicati due principi complementari. Da un lato il **Single Responsibility Principle**, separando la generazione delle mosse geometricamente valide (responsabilità delle classi *Piece*) dalla loro validazione rispetto alle regole del gioco (responsabilità del *RuleEngine*). Si realizza, così, una **Single Source of Truth** per il regolamento dello Xiangqi, ogni modifica alle regole del gioco interesserà un unico componente, evitando duplicazioni e garantendo un comportamento coerente.

RuleEngine come servizio stateless

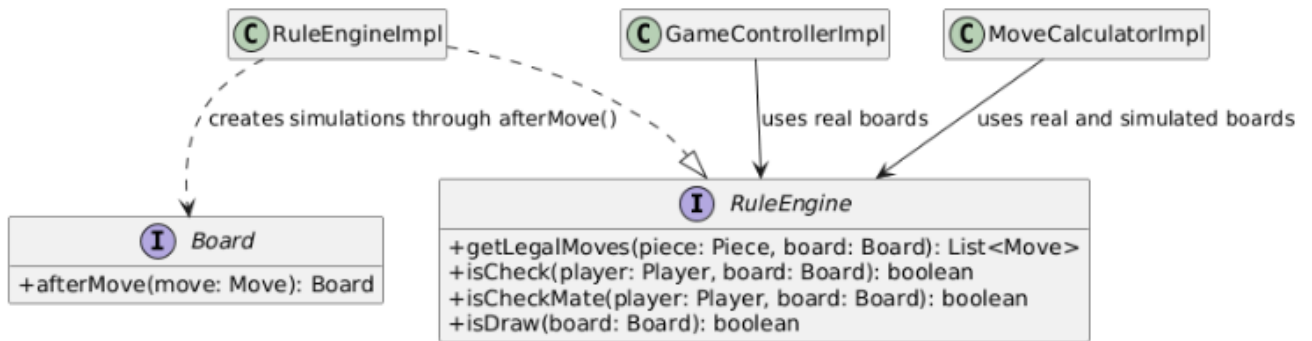


Figura 2.11: Diagramma UML che evidenzia il *RuleEngine* come servizio stateless.

Problema: La verifica della legalità di una mossa richiede di valutare gli effetti sullo stato della partita. A questo si aggiunge un ulteriore requisito imposto dal *MoveCalculator*: durante la ricerca della mossa migliore esso non si limita ad analizzare la configurazione corrente, ma costruisce un albero di ricerca in cui ogni mossa genera una nuova configurazione della scacchiera, dalla quale vengono simulate ulteriori mosse. Il *RuleEngine* deve quindi essere in grado di verificare la legalità delle mosse non solo sulla partita reale, ma anche su una successione di stati ipotetici indipendenti tra loro.

Soluzione: Il *RuleEngine* è stato progettato come un servizio stateless: non conserva alcun riferimento al *GameModel* né allo stato corrente della partita. Ogni operazione richiede esplicitamente la *Board* sulla quale effettuare le verifiche. Per determinare la legalità di una mossa viene applicata la mossa in analisi ad una copia della board tramite *board.afterMove(..)*, sulla quale vengono eseguiti i controlli di scacco e della regola del Flying General. La board originale non viene mai modificata: ogni simulazione produce una nuova configurazione indipendente che può essere utilizzata come punto di partenza per simulazioni successive. In questo modo lo stesso *RuleEngine* può essere utilizzato sia durante il normale svolgimento della partita sia dal *MoveCalculator* durante l'esplorazione dell'albero di ricerca, applicando il principio di **Stateless Service** e mantenendo il componente indipendente dallo stato della partita.

Persistenza disaccoppiata dal modello di dominio vivo

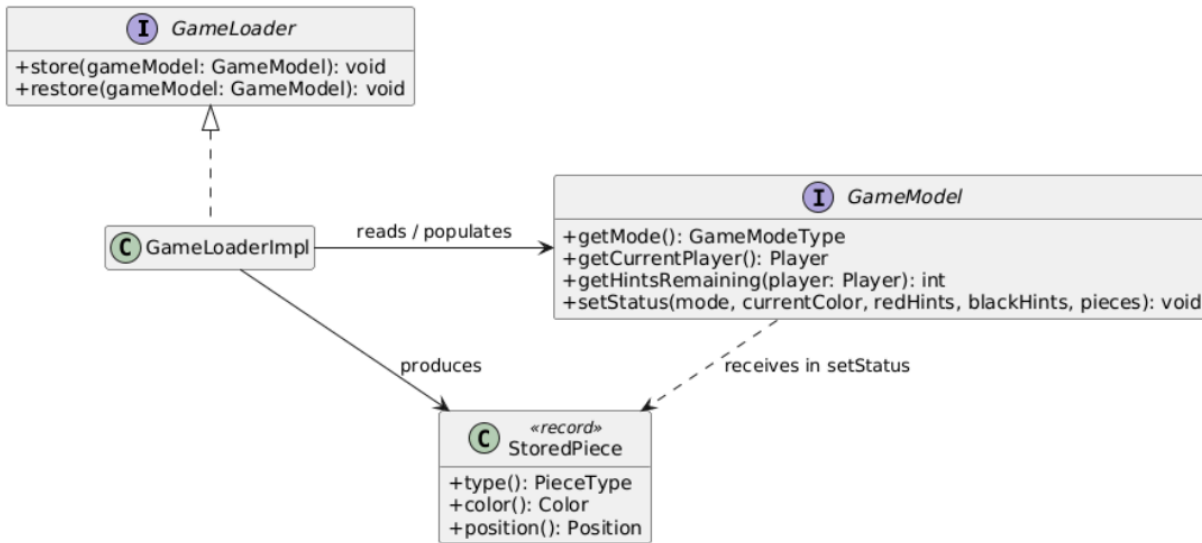


Figura 2.12: Diagramma UML della gestione della persistenza.

Problema: Salvare e ricaricare una partita richiede di trasferire su file lo stato essenziale del gioco: configurazione della scacchiera, giocatore di turno e suggerimenti rimanenti. Tuttavia, gli oggetti del dominio non rappresentano semplici dati persistenti. *Piece* mantiene un riferimento al proprio *Player* e contiene informazioni che non appartengono allo stato persistente, come *initialValue* e *currentValue*, le quali vengono determinate dalla logica del dominio e possono essere ricalcolate durante la partita. Se il *GameLoader* avesse ricostruito direttamente tali oggetti, avrebbe dovuto conoscere sia i dettagli costruttivi del dominio sia le relazioni tra le sue entità, con il rischio di creare riferimenti incoerenti rispetto allo stato gestito dal *GameModel*. Analogamente, ricostruire i *Player* all'interno del *GameLoader* avrebbe richiesto di replicare logiche proprie del modello, come la determinazione del giocatore umano o del bot in funzione del *GameModeType*.

Soluzione: Per separare questi aspetti è stato adottato il pattern **Data Transfer Object (DTO)** tramite *StoredPiece*, un record immutabile composto solo da *PieceType*, *Color* e *Position*. Questo oggetto rappresenta esclusivamente i dati necessari a descrivere un pezzo salvato, senza contenere comportamento né riferimenti a istanze reali di *Player*. In fase di ripristino (*restore()*), il *GameLoader* ricostruisce una lista di *StoredPiece* e la passa al *GameModel* tramite *setStatus()*, insieme al *GameModeType*, al colore del giocatore corrente e agli hint rimanenti. È poi il modello, a ricostruire le effettive istanze di *Piece*, associandole alle istanze di *Player* appartenenti alla partita e inizializzando tutti gli attributi che non fanno parte dello stato persistente. In questo modo il *GameLoader* rimane responsabile solo della traduzione tra file e dati persistenti, mentre il *GameModel* mantiene la responsabilità di ricostruire uno stato di gioco consistente. Il DTO favorisce quindi il **Single Responsibility Principle** e **Low Coupling**, evitando che il formato di salvataggio dipenda dalla struttura interna degli oggetti del dominio.

Politica di salvataggio guidata dallo stato della partita

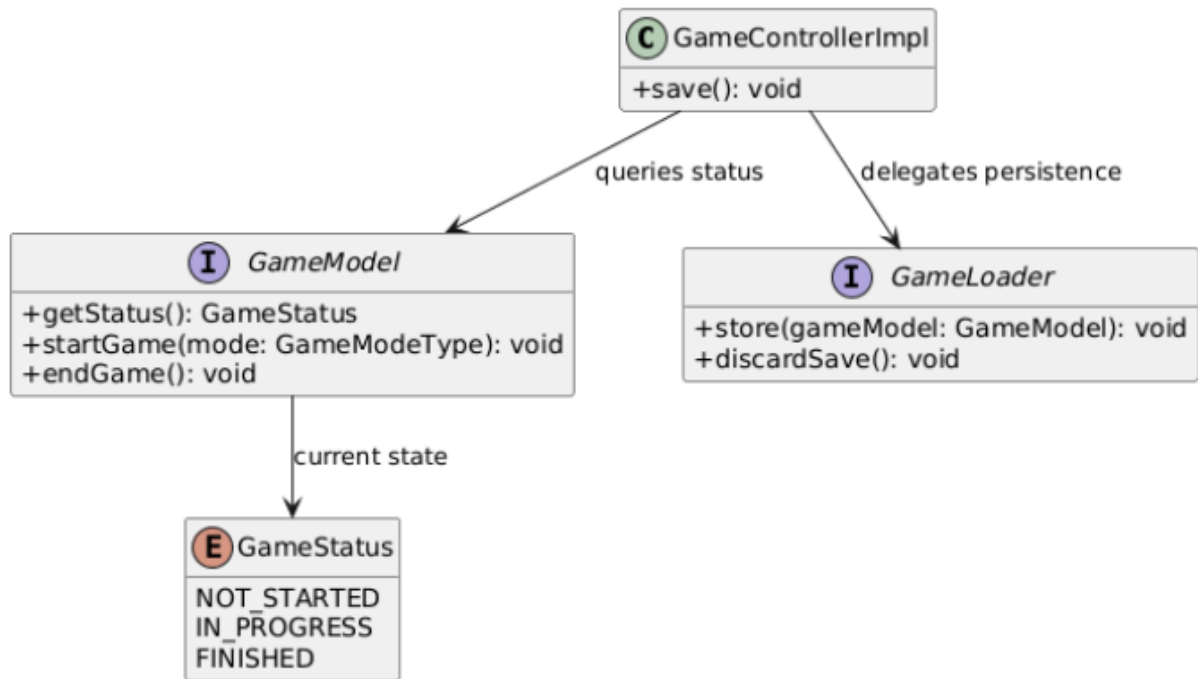


Figura 2.13: Diagramma UML della politica di persistenza guidata dallo stato della partita.

Problema: La chiusura dell'applicazione non richiede sempre la stessa operazione di persistenza. Se una partita è in corso, lo stato deve essere salvato per poter essere ripristinato successivamente. Se invece la partita è terminata, mantenere il file di salvataggio porterebbe a proporre all'utente il ripristino di una partita già conclusa. Infine, se nessuna partita è stata avviata, non deve essere modificato un eventuale salvataggio preesistente. Una gestione basata su un semplice valore booleano non sarebbe sufficiente, perché non distinguerebbe correttamente questi tre casi.

Soluzione: È stata introdotta l'enumerazione *GameStatus*, che definisce lo stato corrente del gioco. Il *GameModel* lo aggiorna internamente durante il ciclo di vita della partita: l'avvio e il ripristino portano il gioco nello stato *IN_PROGRESS*, mentre la conclusione lo porta a *FINISHED*. Il metodo *save()* del *GameController* decide quindi quale operazione eseguire in base allo stato corrente.

- *IN_PROGRESS* = salva la partita corrente;
- *FINISHED* = elimina l'eventuale salvataggio precedente;
- *NOT_STARTED* = non modifica alcun file.

La soluzione applica il principio **make illegal states unrepresentable** modellando il ciclo di vita della partita come un insieme finito di stati espliciti che corrispondono ad un comportamento ben definito e vengono eliminate configurazioni logicamente inconsistenti.

Capitolo 3 — Sviluppo

3.1 Testing automatizzato

Le principali componenti della logica di gioco sono state sottoposte a testing automatizzato al fine di verificare il corretto funzionamento dell'applicazione e prevenire eventuali regressioni introdotte durante lo sviluppo.

Per la realizzazione della test suite è stata utilizzata la libreria **JUnit** e **Mockito**. Tutti i test vengono eseguiti automaticamente e non richiedono alcun intervento da parte dell'utente.

I componenti e le funzionalità sottoposti a testing automatico comprendono:

Piece → Per ciascuna delle sette tipologie di pezzo sono verificate le rispettive regole di movimento, inclusi i casi di cattura, blocchi da parte di pezzi alleati o avversari, il rispetto dei limiti della scacchiera, del fiume e del palazzo, nonché i principali casi limite ai bordi della scacchiera.

GameModel → Viene verificato il corretto ciclo di vita della partita, comprendendo l'inizializzazione dello stato di gioco, la gestione del turno, il conteggio degli hint, la cattura dei pezzi, la produzione di copie indipendente e le principali operazioni di gestione dello stato della partita.

Board → Viene verificato brevemente il metodo *getPieceAt()*, che permette di ottenere un pezzo della *Board* a partire dalla sua posizione.

GameState → Viene creato un *GameState* di partenza, sul quale viene applicata una mossa, ottenendo un nuovo *GameState*. Si verifica che il pezzo sia stato spostato nella posizione corretta e che il *currentPlayer* sia stato aggiornato al giocatore successivo.

GameViewImpl, BoardPanel e MenuPanel → Queste classi sono state testate gradualmente, in modo manuale, nella bootstrap class *XiangqiApplication*. Per far ciò sono state usate delle implementazioni simulate come *FakePiece*, *FakePlayer* etc.... I test veri e propri non sono presenti nel sorgente finale.

MoveCalculator → Vengono verificate le funzioni principali della classe, come la scelta della mossa migliore, il calcolo della *boardScore* e implicitamente anche il corretto funzionamento dei valori dei pezzi. In particolare vengono effettuati i seguenti casi:

- Calcolo della *boardScore* solo sui pezzi di un singolo giocatore;
- Valutazione dei valori di tutti i pezzi sotto determinate situazioni;
- Calcolo del bonus per minaccia ai pezzi avversari e protezione dei pezzi alleati;
- Valutazione dei valori dei pezzi sotto minaccia;
- Correttezza nella scelta della mossa migliore.

GameLoader → I test sono stati progettati con *GameModel* “mockato” tramite Mockito, isolando la logica di persistenza dal resto del dominio, e utilizzando *@TempDir* di JUnit per operare su file temporanei che vengono automaticamente cancellati al termine di ogni test, garantendo completa autonomia rispetto al filesystem del sistema di sviluppo. Viene verificato il

ciclo completo di salvataggio e ripristino (*round-trip*): dopo aver invocato *store()* su un modello con uno stato noto e successivamente *restore()* su un modello mockato, si controlla, tramite *ArgumentCaptor* di Mockito, che *setStatus()* riceva correttamente la modalità di gioco, il colore del giocatore di turno, il numero di suggerimenti rimanenti per entrambi i giocatori e la corretta ricostruzione dei dati persistenti dei pezzi in *StoredPiece*. Inoltre viene testata la gestione del file di persistenza, verificando il comportamento di *hasStoredGame()* in presenza di file inesistenti, vuoti o contenenti dati validi, e il corretto funzionamento di *discardSave()* sia in presenza sia in assenza del file di salvataggio. Infine, sono stati verificati i principali casi di errore durante il caricamento, controllando che venga sollevata un' *IllegalStateException* in presenza di file inesistenti, descrizioni di pezzi incomplete o file troncati rispetto al formato previsto.

RuleEngine → Le principali operazioni sono state sottoposte a test unitari completamente isolati dai componenti collaboratori. Poiché *RuleEngine* dipende dalle interfacce *Board*, *Piece* e *Player*, implementate da altri membri del gruppo, è stato utilizzato **Mockito** per sostituirle con oggetti simulati, verificando esclusivamente la sua logica senza dipendere dalle rispettive implementazioni. I test sono stati costruiti in modo unilaterale, considerando il giocatore rosso come soggetto delle verifiche e il nero come avversario. Tale scelta è motivata dalla simmetria delle regole del gioco: il comportamento del *RuleEngine* non dipende dal colore del giocatore, ma esclusivamente dalla relazione tra giocatore attivo e avversario. La suite copre la rilevazione dello scacco (*isCheck()*), il riconoscimento dello scacco matto (*isCheckMate()*), l'individuazione delle condizioni di patta (*isDraw()*) e il filtraggio delle mosse legali (*getLegalMoves()*), verificando sia gli esiti positivi e negativi delle operazioni, sia casi particolari quali l'assenza di mosse disponibili e l'esclusione delle mosse che lascerebbero il proprio generale sotto scacco. Per i metodi che richiedono la simulazione di una mossa, la board risultante da *board.afterMove()* è stata sostituita mediante mock, consentendo di verificare il comportamento del *RuleEngine* indipendentemente dall'implementazione della simulazione della scacchiera.

3.2 Note di sviluppo

Elencare per punti le feature avanzate di linguaggio/librerie usate, ciascuna con un permalink GitHub al codice scritto in prima persona.

3.2.1 Shenghui Hong

Utilizzo di **Stream API con method reference e toList**: utilizzato nel metodo *copyState()* della classe *GameModelImpl* per copiare in profondità la lista dei pezzi della scacchiera reale.

- <https://github.com/hjleeu-unibo/OOP25-XiangQi/blob/477176a088045298eb070e956389bbbb2b74a2f5/src/main/java/it/unibo/xiangqi/model/impl/GameModelImpl.java#L136>

Uso di **Switch expression**: utilizzato in *PieceFactory* per selezionare la sottoclasse concreta di *Piece* da istanziare in base al *Piecetype*.

- <https://github.com/hjleeu-unibo/OOP25-XiangQi/blob/477176a088045298eb070e956389bbbb2b74a2f5/src/main/java/it/unibo/xiangqi/model/impl/PieceFactory.java#L115>

Utilizzo di **Stream e lambda expressions**:

Usate pervasivamente. Il seguente è un esempio più significativo, utilizzato nel metodo *setStatus()* di *GameModelImpl* per ricostruire la lista dei pezzi a partire dai dati salvati.

- <https://github.com/hjleeu-unibo/OOP25-XiangQi/blob/477176a088045298eb070e956389bbbb2b74a2f5/src/main/java/it/unibo/xiangqi/model/impl/GameModelImpl.java#L154>

3.2.2 Mansour Chabraoui

Uso di lambda expression:

- <https://github.com/hjleeu-unibo/OOP25-XiangQi/blob/477176a088045298eb070e956389bbbb2b74a2f5/src/main/java/it/unibo/xiangqi/view/impl/BoardPanel.java#L99>

3.2.3 Haojie Liu

- Uso di `edu.umd.cs.findbugs.annotations.SuppressFBWarnings`. Esempio in [GameControllerImpl.java:49](#).
- Uso di `Thread`. Esempio in [GameControllerImpl.java:151](#).
- Uso di `java.awt.Font`. Esempio in [Timer.java:40](#).
- Uso di `SwingUtilities.invokeLater` per gestione dei Thread. Esempio in [Timer.java:68](#).
- Uso di `System.currentTimeMillis()` per il timer. Esempio in [GameTimeImpl.java:23](#).
- Uso di **Mockito.mock** per testing. Esempio in [TestMoveCalculator.java:87](#).
- Uso del costrutto **when(...).thenReturn(...)**. Esempio in [TestMoveCalculator.java:98](#).
- Uso di **Mockito.doAnswer**. Esempio in [TestMoveCalculator.java:901](#).
- Uso di **@BeforeEach**. Esempio in [TestMoveCalculator.java:85](#).

3.2.4 Aicha Nidam

- Uso di **Predicate<T>** come parametro funzionale in *RuleEngineImpl*:
 - [Predicate<T>](#)
- Uso di **Stream API con method reference e toList** in *BoardImpl*:
 - [Stream](#)
 - [Method Reference](#)
 - [toList](#)
- Uso di **switch** in *GameControllerImpl*:
 - [switch\(\)](#)
- Uso di **Mockito.mock** per testing in più punti:
 - [Mockito.mock](#)
- Uso del costrutto **when(...).thenReturn(...)** in più punti:
 - [when\(...\).thenReturn\(...\)](#)
- Uso di **@BeforeEach**:
 - [@BeforeEach](#)
- Uso di **Record** in *StoredPiece*:
 - [Record](#)
- Uso di **@TempDir** per il testing di *GameLoader*:
 - [@TempDir](#)

Capitolo 4 — Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Shenghui Hong

Questo progetto rappresenta la mia prima esperienza di sviluppo in un gruppo di lavoro universitario e, per questo motivo, ha avuto per me un valore che va oltre il risultato tecnico ottenuto. All'interno del gruppo mi sono occupato principalmente dello sviluppo del modello di gioco, realizzando la gerarchia dei pezzi e il mio obiettivo è stato quello di fornire un modello solido, coerente e sufficientemente stabile da poter essere utilizzato dagli altri membri del gruppo. Questo ha richiesto un continuo confronto con gli altri componenti del sistema e diversi adattamenti dell'interfaccia del modello, affinché le varie parti del progetto potessero integrarsi correttamente. Analizzando il lavoro svolto, riconosco che la mia parte del progetto presenta ancora alcuni margini di miglioramento. In particolare, ritengo che l'aspetto più critico riguardi la separazione delle responsabilità all'interno del modello di dominio. In un'evoluzione futura mi concentrerei di individuare ulteriori opportunità di riuso riducendo il codice duplicato e migliorando ulteriormente la manutenibilità della soluzione. Un ulteriore aspetto che migliorerei riguarda la funzionalità di resume della partita. Attualmente il ripristino dello stato di gioco non include il tempo residuo dei giocatori: poiché il sistema di timer rappresenta una funzionalità opzionale, durante lo sviluppo è stata data priorità all'integrazione delle funzionalità principali. Di conseguenza, dopo il caricamento di una partita salvata, il timer viene impostato al valore iniziale anziché riprendere dal tempo effettivamente rimanente. In un'evoluzione futura estenderei il meccanismo di serializzazione anche allo stato del timer, così da ottenere un ripristino completo e coerente della partita. Ritengo che questa esperienza mi abbia permesso di comprendere concretamente quanto sia importante progettare componenti stabili e ben definiti quando costituiscono la base su cui lavorano altri membri del gruppo, oltre a rafforzare le capacità di collaborazione e integrazione del codice.

4.1.2 Mansour Chabraoui

Il mio ruolo all'interno del gruppo è stato principalmente quello di occuparsi dell'interfaccia grafica, implementando anche qualche classe del Model.

Durante il lavoro ho applicato la tecnica di "programmazione incrementale", che mi ha permesso di limitare i bug emersi durante la fase di debugging. Tuttavia penso ci sia ancora un margine di miglioramento per quanto riguarda il testing: la parte della view è stata testata in modo completamente manuale, e questi test non sono nemmeno reperibili nella versione finale del sorgente, problema che ho sottovalutato durante l'implementazione.

Sono stato abbastanza attento a ridurre ripetizioni di codice, rispettando il principio "Don't repeat yourself". Mi sono però accorto di non aver usato feature avanzate, limitandomi ad un codice molto semplice.

Come direzioni future di questo progetto penso ad aggiungere nuove modalità.

4.1.3 Haojie Liu

A mio parere, il sistema adattato per la selezione della mossa migliore, implementato nel *MoveCalculator*, è stato una scelta adeguata per un progetto universitario, ma non riesce a soddisfare le esigenze per un progetto nel mondo del lavoro in quanto la logica risulta limitata

superiormente. Infatti ci limitiamo solo ai primi tre passi invece di continuare a monitorare tutte le possibili situazioni da quel momento fino alla fine della partita. Questo difetto causa il bot a scegliere delle mosse stupide in particolari situazioni.

Nonostante la conoscenza di questo limite, ho scelto questo sistema perché è stata modellata man mano andando avanti nel progetto, aggiungendo sempre più controlli, avendo tutto sotto controllo, invece di scegliere un algoritmo complesso già implementato da altri, di cui non conoscevo le caratteristiche.

Per quanto riguarda il timer, implementata coi threads, la difficoltà di implementazione è risultata gestibile. Tuttavia, il margine di miglioramento è ancora lontano.

Riguardo ai lavori futuri, trovare un algoritmo migliore per la scelta della mossa migliore risulta come un upgrade più importante. Oltre a ciò, anche le prestazioni dell'intero sistema risultano migliorabili.

Pur essendo la seconda volta a lavorare in gruppo usando GitHub, all'inizio mi sono trovato davanti a molte difficoltà nell'interazione tra i diversi branches, ma più andando avanti nel tempo, comincio ad abituarci a questa modalità. Ritengo che questa esperienza mi sarà molto utile per il mio percorso futuro.

4.1.4 Aicha Nidam

Il mio contributo al progetto riguarda diversi aspetti della logica di gioco: il confine tra interfaccia e dominio (*InputHandler*, *GameController*), il motore delle regole (*RuleEngine*) e la gestione della persistenza (*GameLoader*, *StoredPiece*). Ho inoltre implementato *afterMove()* della classe *Board*.

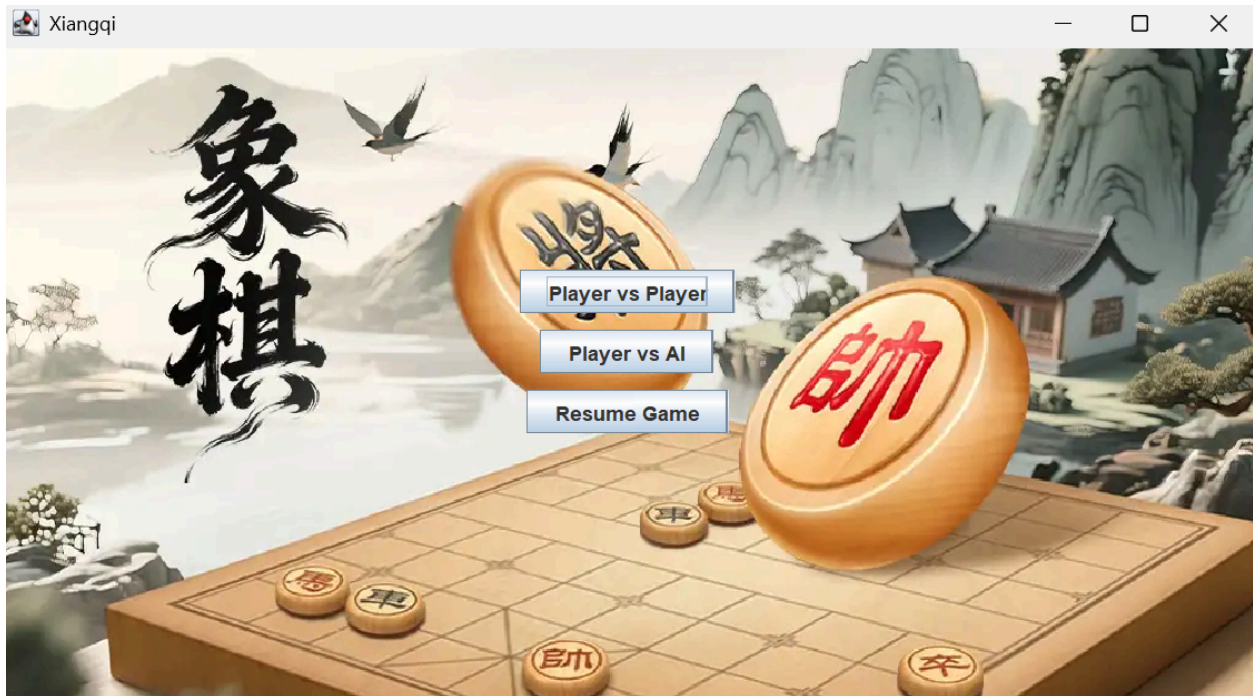
L'aspetto di cui sono maggiormente soddisfatta è la progettazione del *RuleEngine* come servizio stateless. Durante lo sviluppo è emersa una criticità che impediva il corretto funzionamento del turno del bot e del sistema di suggerimenti. La sua risoluzione mi ha portato a ripensare il modo in cui il motore delle regole interagiva con lo stato della partita, ottenendo un componente completamente indipendente dal *GameModel* e riutilizzabile sia durante il normale svolgimento della partita sia nelle simulazioni effettuate dall'intelligenza artificiale.

Sono soddisfatta anche della disciplina mantenuta nell'*InputHandler*, che ho voluto ridotto a un traduttore puro tra eventi della view e azioni di dominio. Mantenerlo essenziale ha permesso di preservare la separazione dei ruoli e di rendere l'intero flusso di gioco testabile senza dipendere dall'interfaccia grafica.

All'interno del gruppo ho lavorato in modo autonomo sulla mia parte, coordinandomi con i compagni per le firme dei metodi condivisi e per la risoluzione del bug di simulazione, che ha coinvolto classi scritte da persone diverse. Questa esperienza mi ha inoltre permesso di acquisire dimestichezza con le classi di testing autonome, imparando a isolare i componenti sotto test dai loro collaboratori tramite l'uso di mock objects oltre a consolidare l'utilizzo degli strumenti di collaborazione di GitHub dalla gestione dei branch e dei merge fino alla risoluzione dei conflitti e più in generale di confrontarmi per la prima volta con le dinamiche di uno sviluppo software di gruppo, che considero uno degli aspetti più formativi dell'intera esperienza.

Appendice A — Guida utente

Avvio dell'applicazione



All'avvio dell'applicazione compare il menu principale, che presenta tre opzioni:

- **Player vs Player:** modalità di gioco a due giocatori umani sullo stesso dispositivo. Il giocatore rosso muove per primo.
- **Player vs Bot:** modalità di gioco contro un avversario controllato dal computer. Il giocatore umano gioca sempre come rosso e inizia la partita.
- **Resume Game:** riprende l'ultima partita interrotta, se presente. Se non esiste alcuna partita salvata, un messaggio a video segnala l'assenza di dati di ripristino.

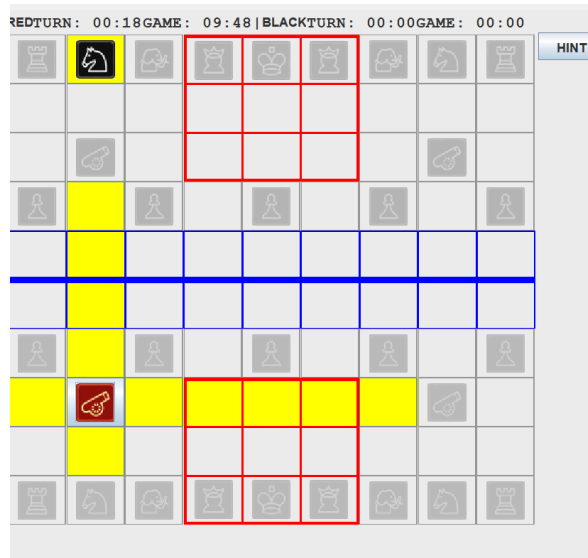
La scacchiera

La scacchiera dello Xiangqi è composta da 9 colonne e 10 righe. Sono visivamente delimitate due aree particolari:

- La **fascia centrale**, contornata in **blu**, rappresenta il **fiume**: una regione che alcuni pezzi non possono attraversare o che ne modifica le regole di movimento.
- Le due **aree contornate in rosso** alle estremità della scacchiera rappresentano i **palazzi** dei due giocatori: al loro interno si muovono esclusivamente il Generale e i Consiglieri.

Per effettuare una mossa

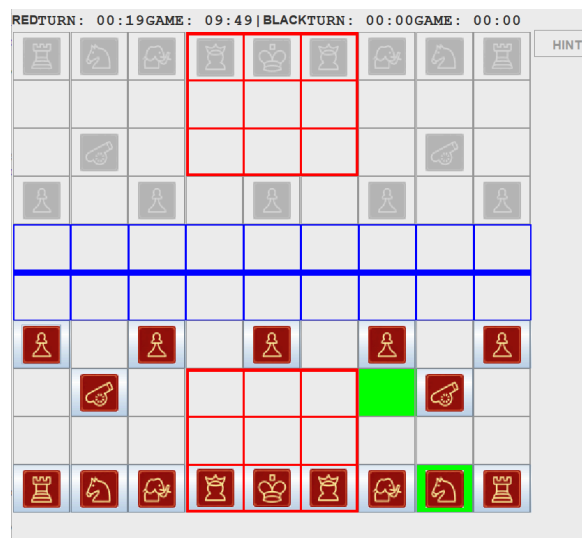
Per muovere un pezzo:



1. Cliccare sulla casella del pezzo che si desidera spostare. Le **caselle di destinazione ammesse** si evidenziano in **giallo**.
2. Cliccare su una delle caselle gialle per confermare la mossa.
È fondamentale notare che non è possibile selezionare caselle al di fuori di quelle evidenziate.

Se si desidera cambiare pezzo dopo la selezione, è sufficiente ri-cliccare sulla casella del pezzo appena selezionato per deselectionarlo, quindi scegliere un altro pezzo.

Suggerimenti (Hint)



Ogni giocatore ha a disposizione un massimo di **tre suggerimenti** per partita. Il suggerimento può essere richiesto durante il proprio turno cliccando il bottone **Hint**. Una volta cliccato:

- La casella del pezzo consigliato si evidenzia in **verde**.
- La casella di destinazione suggerita si evidenzia anch'essa in **verde**.

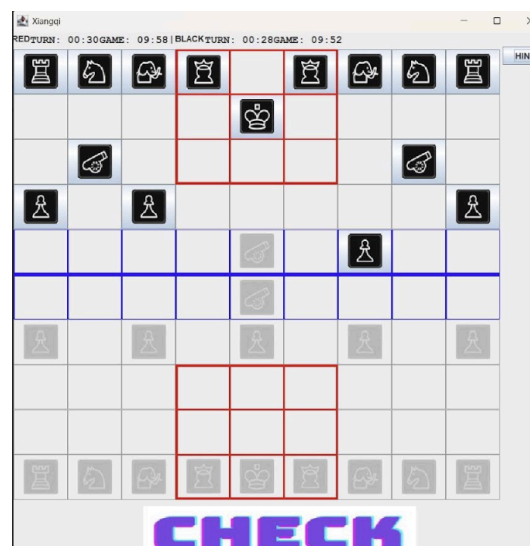
Quando i tre suggerimenti disponibili sono stati utilizzati, il bottone Hint non è più accessibile per il resto della partita.

Timer di gioco

Nella parte superiore dell'interfaccia sono visualizzati due timer per ciascun giocatore:

- **Timer di turno: 30 secondi** è la durata massima consentita per il singolo turno del giocatore. Allo scadere del timer, automaticamente si dichiara la vittoria dell'avversario.
- **Timer di partita:** la durata massima complessiva di una partita, è pari a **dieci minuti**.

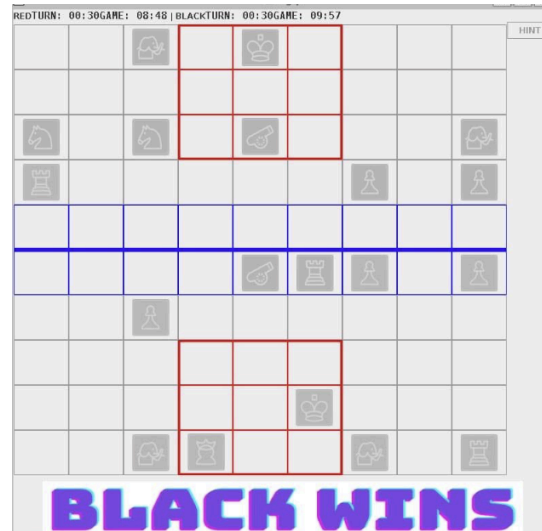
Situazione di scacco



Quando il proprio Generale è sotto attacco, sotto la scacchiera compare un **banner di scacco** che segnala la situazione. In questo caso il numero di mosse disponibili si riduce, perché sono ammesse solo le mosse che permettono di uscire dallo scacco (spostando il Generale, catturando il pezzo attaccante, oppure interponendo un pezzo tra il Generale e la minaccia).

Fine della partita

Al termine della partita compare un banner che dichiara il vincitore, riportando il colore del giocatore vittorioso (ad esempio *Red Wins* per il giocatore rosso). Nel caso di pareggio appare la corrispondente scritta *Draw*.



Salvataggio automatico e ripristino

Il salvataggio della partita è **automatico** e segue queste regole:

- **Chiusura dell'applicazione con la X durante una partita in corso:** lo stato attuale viene salvato automaticamente e potrà essere ripristinato al successivo avvio tramite l'opzione *Resume Game*.
- **Nuova interruzione di un'altra partita:** il salvataggio precedente viene sovrascritto: rimane memorizzata soltanto l'ultima partita interrotta.
- **Partita conclusa fino alla vittoria o pareggio:** l'eventuale salvataggio precedente viene automaticamente eliminato.